Project Structure

This project follows a modular, environment-isolated Terraform architecture designed to support multiple environments (staging and production) while avoiding code duplication and maintaining a clean GitHub workflow.

High-Level Directory Layout

```
terraform/
├── root/
│   ├── main.tf
│   ├── provider.tf
│   └── variables.tf
│
├── modules/
│   ├── network/
│   │   ├── main.tf
│   │   ├── variables.tf
│   │   └── outputs.tf
│   │
│   ├── bastion/
│   │   ├── main.tf
│   │   ├── variables.tf
│   │   ├── outputs.tf
│   │   └── user_data.sh
│   │
│   └── rds/
│       ├── main.tf
│       ├── variables.tf
│       └── outputs.tf
│
└── envs/
    ├── staging/
    │   ├── backend.hcl
    │   └── terraform.tfvars
    │
    └── production/
        ├── backend.hcl
        └── terraform.tfvars
```

- **Root Module**
  - Main.tf
    - Main Terraform file
    - Calls all modules: network, bastion, rds

- Provider.tf
    - Defines Terraform version constraints
    - Configures AWS provider
    - Declares remote backend (s3)
    - Applies default resource tags
- Variables.tf
    - Declares all variables required by the root module (main.tf)
    - Receives values from environment-specific terraform.tfvars

➢ **<u>Modules</u>**

- Network Module
    - Create VPC
    - Create Private & Public Subnets
    - Create Internet Gateway
    - Create Private & Public Route Table & Associations

- Bastion Module
    - Create Security Group for Bastion host to allow SSH from user's IP
    - Create EC2 Instance placed in Public Subnet with bootstrapped user-data

- RDS Module
    - Create Security Group allow MySQL access only from bastion
    - MySQL database username/password stored in Secret Manager
    - Create Private database subnet group
    - Create MySQL RDS Instance placed in Private Subnet

➢ **<u>Environment Configuration</u>**

- Each environment uses a separate Terraform backend, variable file, and GitHub Environment, ensuring state, credentials, and deployments are fully isolated.
- Staging and production environments share the same Terraform codebase, while isolation is achieved through: Terraform backends & terraform.tfvars
- Backend.hcl
    - S3 bucket for Terraform state
    - State file key
    - AWS region
    - DynamoDB table for locking
- Terraform.tfvars
    - Provides environment-specific values for: network, bastion & rds modules

Terraform Deployment Patterns

1. **Local Deployment**
   - Staging Deployment:
     - Root folder: cd .\terraform\root\
     - $env:AWS_PROFILE="staging"; terraform init -backend-config="../envs/staging/backend.hcl"
     - $env:AWS_PROFILE="staging"; terraform plan -var-file="../envs/staging/terraform.tfvars"
     - $env:AWS_PROFILE="staging"; terraform apply -auto-approve -var-file="../envs/staging/terraform.tfvars"

   - Production Deployment:
     - Root folder: cd .\terraform\root\
     - $env:AWS_PROFILE="production"; terraform init -backend-config="../envs/production/backend.hcl"
     - $env:AWS_PROFILE="production"; terraform plan -var-file="../envs/production/terraform.tfvars"
     - $env:AWS_PROFILE="production"; terraform apply -auto-approve -var-file="../envs/production/terraform.tfvars"

2. **GitHub Action Workflow**
   - Pull Request: terraform fmt, validate, plan (staging)
   - Merge to main: terraform apply (staging)
   - v*.*.* tag: terraform apply (production)
   - Credential for Staging & Production accounts are stored in GitHub Secrets
   - Terraform state files and terraform.tfvars are excluded from version control to prevent accidental exposure of sensitive data.

Manual Testing Steps

- **Prerequisites**
  - AWS accounts for **staging** and **production** are accessible
  - Terraform infrastructure has been deployed successfully
  - SSH key pair for bastion host is available
  - Your public IP is allowed in the bastion security group

- **Bastion-Host Connect**
  - Run this command from the cmd: ssh -i "<path-to-my-ssh-key.pem>" ubuntu@<ec2-public-dns>

- **MySQL Connect**
  - Now, using your Bastion-host, connect to AWS MySQL RDS
  - Command to connect: mysql -h <mysql-rds-endpoint> -P 3306 -u <username> -p --ssl-ca=/certs/global-bundle.pem --ssl-mode=VERIFY_CA appdb
  - After this, you will enter to the mysql terminal.